

1. The D&C; Paradigm

Divide and Conquer solves a problem by breaking it into subproblems, solving each recursively, then combining the results. Every D&C; algorithm has exactly three phases:

Phase	What it does
Divide	Split the problem instance into smaller subproblems.
Conquer	Recursively solve each subproblem (base case: solve directly).
Combine	Merge the subproblem solutions into the final answer.

2. Binary Search

Problem: Find a target element in a sorted array.

Phase	Action
Divide	Check the middle element against the target.
Conquer	Recursively search only 1 of the 2 halves.
Combine	Trivial — the recursive result is the final result.

Recurrence:

$$T(n) = 1 \cdot T(n/2) + \Theta(1)$$

Apply Master Theorem Case 2: $n^{\log_b(a)} = n^{\log_2(1)} = n^0 = 1$, so $f(n) = \Theta(1) = \Theta(n^0 \lg n)$. $\rightarrow T(n) = \Theta(\lg n)$

3. Merge Sort

Problem: Sort an array of n elements.

Phase	Action
Divide	Trivial — split array into two halves of size $n/2$.
Conquer	Recursively sort both halves (2 subproblems).
Combine	MERGE the two sorted halves — this takes $\Theta(n)$ time.

MERGE subroutine (key idea):

Maintain two pointers, one into each sorted half. Repeatedly pick the smaller front element and append it to the output array C . When one half is exhausted, copy the remainder of the other. Time: $\Theta(n)$. Issue: this is an *out-of-place* algorithm — it requires $O(n)$ extra space.

Recurrence:

$$T(n) = 2 \cdot T(n/2) + \Theta(n)$$

Apply Master Theorem Case 2: $n^{\log_b(a)} = n^{\log_2(2)} = n^1 = n$, so $f(n) = \Theta(n) = \Theta(n^1 \lg n)$. $\rightarrow T(n) = \Theta(n \lg n)$

Pseudocode skeleton:

```

MERGE-SORT(A[1..n]):
    if n = 1: return
    A1 = MERGE-SORT(A[1..n/2])
    A2 = MERGE-SORT(A[n/2+1..n])
    A = MERGE(A1, A2)
    
```

4. Powering a Number

Problem: Compute a^n for $n \in \mathbb{N}$. Naive repeated multiplication takes $\Theta(n)$.

D&C; recurrence:

$$a^n = a^{n/2} \cdot a^{n/2} \text{ if } n \text{ is even}$$

$$a^n = a^{(n-1)/2} \cdot a^{(n-1)/2} \cdot a \text{ if } n \text{ is odd}$$

$T(n) = T(n/2) + \Theta(1) \rightarrow$ same recurrence as binary search $\rightarrow T(n) = \Theta(\lg n)$

5. Master Theorem — Quick Reference

For $T(n) = a \cdot T(n/b) + f(n)$, compare $f(n)$ with $n^{\log_b(a)}$:

Case	Condition	Result
1	$f(n) = O(n^{\log_b(a) - \epsilon})$ for some $\epsilon > 0$	$T(n) = \Theta(n^{\log_b(a)})$
2	$f(n) = \Theta(n^{\log_b(a)} \cdot \lg^k n)$, $k \geq 0$	$T(n) = \Theta(n^{\log_b(a)} \cdot \lg^{k+1} n)$
3	$f(n) = \Omega(n^{\log_b(a) + \epsilon})$ and regularity holds	$T(n) = \Theta(f(n))$

6. Other Notable D&C; Algorithms

The following algorithms all follow the Divide-Conquer-Combine pattern:

Algorithm	Time Complexity	Key idea
Binary Search	$\Theta(\lg n)$	Halve search space each step
Merge Sort	$\Theta(n \lg n)$	Split + linear-time merge
Powering (fast)	$\Theta(\lg n)$	Square-and-multiply
Strassen Matrix Mult.	$\Theta(n^{\lg 7}) \approx \Theta(n^{2.81})$	7 recursive multiplications instead of 8
Quicksort	$\Theta(n \lg n)$ avg	Partition around pivot, sort sub-arrays
FFT	$\Theta(n \lg n)$	Polynomial evaluation at roots of unity
Closest Pair	$\Theta(n \lg n)$	Divide plane, combine strip candidates

Key takeaway: D&C; algorithms are analysed using recurrences. The Master Theorem gives a fast closed-form solution for most cases. Divide and Conquer consistently produces $O(n \lg n)$ or better algorithms for problems that naive approaches solve in $O(n^2)$ or $O(n)$.